1. Event:

Change in the state of an object is known as event i.e. event describes the change in state of source. Events are generated as result of user interaction with the graphical user interface components. For example, clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from list, scrolling the page are the activities that causes an event to happen. Events may also occur that are not directly caused by interactions with a user interface. For example, an event may be generated when a timer expires, software or hardware failure occurs, or an operation is completed. Most of the events relating to the GUI for a program are represented by classes defined in the package java.awt.event. This package also defines the listener interfaces for the various kinds of events that it defines. The package javax.swing.event defines classes for events that are specific to Swing components.

- **Types of Event:** The events can be broadly classified into two categories:
  - **Foreground Events** - Those events which require the direct interaction of user are known as foreground events. They are generated as consequences of a person interacting with the graphical components in Graphical User Interface. For example, clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from list, scrolling the page etc.
  - **Background Events** - Those events that does not require the interaction of end user are known as background events. For example: Operating system interrupts, hardware or software failure, if timer expires, an operation completion, etc.

The table below shows most of the important event classes and their description:

Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or losses keyboard focus.
ItemEvent	Generated when a checkbox or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseWheelEvent	Generated when the mouse wheel is moved.

TextEvent	Generated when the value of text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

- **Event Sources**

A source is an object that generates an event. This occurs when the internal state of that object changes in some way. Sources may generate more than one type of event. The table below shows most of the important event sources and their description:

Event Sources	**Description**
Button	Generates action events when the button is pressed.
Checkbox	Generates item events when the checkbox is selected or deselected.
Choice	Generates item events when the choice is changed.
Menu Item	Generates action events when the menu is selected; generates item events when a checkable menu item is selected or deselected.
List	Generates action events when an item is double-clicked; generates item events when an item is selected or deselected.
Scrollbar	Generates adjustment events when the scroll bar is manipulated.
Text Components	Generates text events when the user enters the character.
Window	Generates window events when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

- **Event Listeners:**

To react to an event, we implement appropriate event listener interfaces. A listener is an object that is notified when an event occurs. It has two requirements. First, it must have been registered with one or more sources to receive notifications about specific types of events. Second, it must implement methods to receive and process these notifications. We use addTypeListener() method to register the event listener with the source. Similarly, we can also unregister a listener by using removeTypeListener() method. The table below shows most of the important event listener and their description:

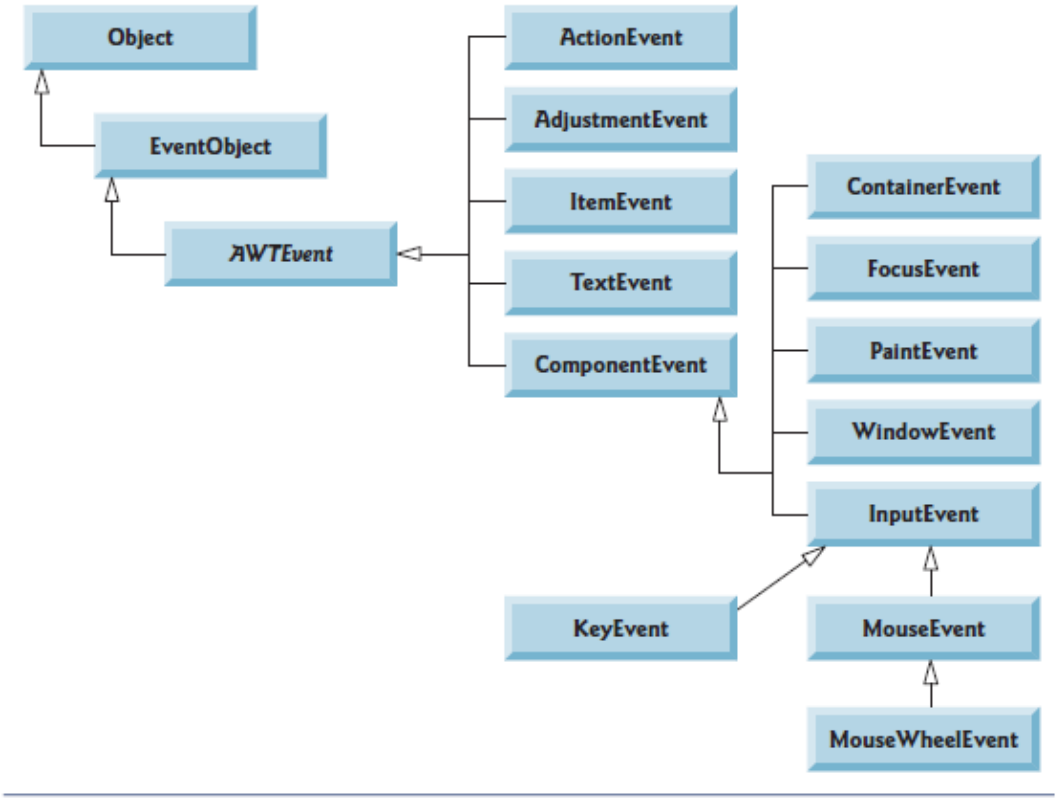
Interface	**Description**
ActionListener	Declares one method to receive action events. void actionPerformed(ActionEvent e)
AdjustmentListener	Declares one method when a scrollbar is manipulated.

	void adjustmentValueChanged (AdjustmentEvent e)
ComponentListener	Declares four methods to recognize when a component is hidden, moved, resized, or shown. void componentResized(ComponentEvent e) void componentMoved(ComponentEvent e) void componentShown(ComponentEvent e) void componentHidden(ComponentEvent e)
ContainerListener	Declares two methods to recognize when a component is added to or removed from a container. void componentAddedd(ContainerEvent e) void componentRemovedd(ContainerEvent e)
FocusListener	Defines two methods to recognize when a component gains or loses keyboard focus. void focusGained(FocusEvent e) void focusLost(FocusEvent e)
ItemListener	Defines one method to recognize when a check box or list item is clicked, when a choice selection is made, or when a checkable menu item is selected or deselected. void itemStateChanged(ItemEvent e)
KeyListener	Defines three method to recognize when a key is pressed, released, or typed. void keyPressed(KeyEvent e) void keyReleased(KeyEvent e) void keyTyped(KeyEvent e)
MouseListener	Defines five method to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released. void mouseClicked(MouseEvent e) void mouseEntered(MouseEvent e) void mouseExited(MouseEvent e) void mousePressed(MouseEvent e) void mouseReleased(MouseEvent e)
MouseMotionListener	Defines two method to recognize when the mouse is dragged or moved. void mouseDragged(MouseEvent e) void mouseMoved(MouseEvent e)
MouseWheelListener	Defines one method to recognize when the mouse wheel is moved. void mouseWheelMoved(MouseWheelEvent e)
TextListener	Defines one method to recognize when a text value changes.

	void textChanged(TextEvent e)
WindowFocusListener	Defines two method to recognize when a window gains or loses input focus. void windowGainedFocus(WindowEvent e) void windowLostFocus(WindowEvent e)
WindowListener	Defines seven method to recognize when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit. void windowActivated(WindowEvent e) void windowDeactivated(WindowEvent e) void windowClosed(WindowEvent e) void windowClosing(WindowEvent e) void windowOpened(WindowEvent e) void windowIconified(WindowEvent e) void windowDeiconified(WindowEvent e)

2. AWT Event Hierarchy

Figure below illustrates a hierarchy containing many event classes from the package **java.awt.event**. These event types are used with both AWT and Swing components. Additional event types that are specific to Swing GUI components are declared in package **javax.swing.event.**



3. Semantics and Low Level Events in AWT:

There are many different kinds of events to which our program need to respond. For example: from menus, buttons, from the mouse, from the keyboard, etc. In order to have a structured approach to handling events, these events can be broken down into subsets. At the topmost level, there are two broad categories of events in Java:

- **Low-level Events** – These are events that arise from the keyboard or from the mouse, or events associated with operations on a window. KeyEvent, FocusEvent, MouseEvent, WindowEvent, ComponentEvent and ContainerEvent are low level events. Mouse and key events, both of which result directly from user input are also low-level events. The meaning of a low-level event is something like 'the mouse was moved', 'this window has been closed' or 'this key was pressed'.
- **Semantic Events** – These are specific component-related events such as pressing a button by clicking it to cause some program action, or adjusting a scrollbar. ItemEvent, TextEvent, AdjustmentEvent and ActionEvent are related with a particular type of component so they are semantic events. The meaning of a semantic event is typically along the lines of: 'the OK button was pressed', or 'the Save menu item was selected'. Each kind of component, a button, or a menu item, can generate a particular kind of semantic event.

These two categories looks similar in some way. If we click a button, we create a semantic event as well as a low level event. The click produces a low-level event object in the form of 'the mouse was clicked' as well as a semantic event 'the button was pushed'. In fact it produces more than one mouse event. Whether our program handles the low-level events or the semantic event, or possibly both kinds of event, depends on what we want to do.

4. Event Handling:

Event Handling is the mechanism that controls the event and decides what should happen if an event occurs. This mechanism have the code which is known as event handler that is executed when an event occurs. Java Uses the Delegation Event Model to handle the events. This model defines the standard mechanism to generate and handle the events. The Delegation Event Model has the following key participants namely:

- **Source** - The source is an object on which event occurs. Source is responsible for providing information of the occurred event to its handler.

- **Listener** - It is also known as event handler. Listener is responsible for generating response to an event. From java implementation point of view the listener is also an object. Listener waits until it receives an event. Once the event is received, the listener process the event and then returns.

The benefit of this approach is that the user interface logic is completely separated from the application logic. In this model, listener needs to be registered with the source object so that the listener can receive the event notification. This is an efficient way of handling the event because the event notifications are sent only to those listener that want to receive them.

- **Steps involved in event handling**
  - The User clicks the button and the event is generated.
  - Now the object of concerned event class is created automatically and information about the source and the event get populated within same object.
  - Event object is forwarded to the method of registered listener class.
  - The method is executed and returns.

Example 1:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class EventExample extends JFrame implements ActionListener
{
public static void main(String args[])
{
EventExample e1=new EventExample();
JFrame f1=new JFrame("Event handling");
JButton b1=new JButton("Click me");
f1.add(b1);
b1.addActionListener(e1);
f1.setVisible(true);
f1.setSize(400,450);
}
public void actionPerformed(ActionEvent e)
{
JOptionPane.showMessageDialog(null, "The button is clicked");
}
}
```

Example 2:

```
import javax.swing.JOptionPane;       // program uses JOptionPane
public class Addition
{
  public static void main( String[] args )
  {
    // obtain user input from JOptionPane input dialogs
    String firstNumber=JOptionPane.showInputDialog( "Enter first integer" );
    String secondNumber=JOptionPane.showInputDialog( "Enter second integer" );
    // convert String inputs to int values for use in a calculation
    int number1 = Integer.parseInt( firstNumber );
    int number2 = Integer.parseInt( secondNumber );
    int sum = number1 + number2; // add numbers
    // display result in a JOptionPane message dialog
    JOptionPane.showMessageDialog( null, "The sum is " + sum,
      "Sum of Two Integers", JOptionPane.PLAIN_MESSAGE );
  } // end method main
```

} // end class Addition

Output